

Biomedical Engineering 580.483
Annotate a Genome
Spring 2026

Due Date: 11:59 pm on 04/01/2026
Total Points: 120

Lecture 4 Exercises: PRS

Polygenic Risk Score (PRS) Analysis

- Concept: Instead of looking at individual SNPs, PRS aggregates thousands of small genetic effects into a single risk score for an individual.

- The Workflow:

- **Discovery Phase:** Use the provided summary stats from a simulated Discovery cohort.
- **Target Phase:** Calculate scores for individuals in our simulated cohort by multiplying their risk alleles by those weights.

$$PRS_i = \sum_{j=1}^M \beta_j G_{ij}$$

- PRS_i : Polygenic Risk Score for individual i
- β_j : The effect size (weight) of risk allele at SNP j , derived from a discovery GWAS.
- G_{ij} : The dosage genotype (0, 1, or 2) of the risk allele for individual i .
- M : Total number of SNPs included.

In a typical PRS study, your Discovery cohort is likely to be from a publicly available dataset such as UK Biobank. What you have to work with will be "GWAS summary stats". For this assignment, we've provided "GWAS summary stats" for a simulated Discovery cohort with the same binary phenotype as our target cohort and for which the same SNP panel was used.

https://karchinlab.org/share/aag_spr2026/lecture_4_exercises/PRS/discovery_gwas_sim_sumstats.tsv.gz

You've already QC'd the target cohort in your GWAS exercise. Now you need to do summary-state QC and harmonization of Discovery and target cohorts.

General Instructions for submission:

For each step, please include the requested output files in a .zip format and upload to gradescope.

OR

Upload all output files to a public github repo and include the path to the repo in your gradescope submission.

For each plot: include (i) code (R or Python) and (ii) the plot.

1) Discovery summary-stat QC (15 pts)

a. Column/format validation

Verify required fields exist: SNP, A1, A2, BETA, SE, P, N, CHR,BP

All required columns are present.

```
import pandas as pd
import numpy as np

# Load the data
print("Loading GWAS summary stats...")
df = pd.read_csv('/Users/kmlander0s/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/data/exercise4/discovery_gwas_sim_sumstats.tsv.gz', sep='\t')

initial_rows = len(df)
print(f"Initial row count: {initial_rows}\n")

# -----
# 1a. Column/format validation
```

```
# -----
required_cols = ['SNP', 'A1', 'A2', 'BETA', 'SE', 'P', 'N', 'CHR', 'BP']
missing_cols = [col for col in required_cols if col not in df.columns]

if missing_cols:
    print(f"1a. Missing columns: {missing_cols}")
else:
    print("1a. All required columns are present.\n")
```

b. Missingness check:

Report number of rows with any missing SNP/A1/A2/BETA/SE/P/N; remove them.

Initial row count: 1,462,262

Rows removed due to missing data: 1,989

Rows remaining: 1,460,273

```
# -----
# 1b. Missingness check
# -----
# Drop rows with missing values in the critical columns
df_clean = df.dropna(subset=['SNP', 'A1', 'A2', 'BETA', 'SE', 'P', 'N'])
missing_dropped = initial_rows - len(df_clean)

print(f"1b. Missingness check:")
print(f"Rows removed due to missing data: {missing_dropped}")
print(f"Rows remaining: {len(df_clean)}\n")
```

c. Range checks:

- Confirm $0 \leq P \leq 1$

- Confirm $SE > 0$

- flag extreme |BETA| outliers

Rows removed (P or SE out of range): 1,240

Flagged extreme |BETA| outliers (> 3 SDs): 28,471

```
# -----
# 1c. Range checks
# -----
# Confirm  $0 \leq P \leq 1$  and  $SE > 0$ 
valid_p = (df_clean['P'] >= 0) & (df_clean['P'] <= 1)
valid_se = df_clean['SE'] > 0

# Apply the filters
pre_range_count = len(df_clean)
df_clean = df_clean[valid_p & valid_se]
range_dropped = pre_range_count - len(df_clean)
```

```

print(f"1c. Range checks:")
print(f"Rows removed (invalid P or SE): {range_dropped}")

# Flag extreme |BETA| outliers
# (Using > 3 Standard Deviations as the definition of "extreme")
beta_mean = df_clean['BETA'].mean()
beta_std = df_clean['BETA'].std()
extreme_beta_mask = np.abs(df_clean['BETA'] - beta_mean) > (3 * beta_std)
extreme_betas = df_clean[extreme_beta_mask]

print(f"Flagged {len(extreme_betas)} extreme |BETA| outliers (> 3 SDs).")
print()

```

d. Duplicate SNP IDs:

Count duplicates and drop any duplicated values (keep the smallest P).

Found 8,716 duplicated SNP rows; dropped 4,358 duplicate records (kept smallest P-value).

Final cleaned row count: 1,454,675

```

# -----
# 1d. Duplicate SNP IDs
# -----
# Count all rows that share a SNP ID with another row
duplicate_count = df_clean.duplicated(subset=['SNP'], keep=False).sum()
print(f"1d. Duplicate SNP IDs:")
print(f"Found {duplicate_count} duplicated SNP rows.")

# Sort by P-value (ascending) so the smallest P is at the top
df_clean = df_clean.sort_values('P')

# Drop duplicates, keeping the first (which has the smallest P due to sorting)
pre_dedup_count = len(df_clean)
df_clean = df_clean.drop_duplicates(subset=['SNP'], keep='first')
dedup_dropped = pre_dedup_count - len(df_clean)

print(f"Dropped {dedup_dropped} duplicate records (kept smallest P).")
print(f"Final cleaned row count: {len(df_clean)}\n")

# -----
# Save Output
# -----
output_file = 'discovery_gwas_sim_sumstats_cleaned.tsv'
df_clean.to_csv(output_file, sep='\t', index=False)

```

Name your QC'd version: discovery_gwas_sim_sumstats_cleaned.tsv
Include the **discovery_gwas_sim_sumstats_cleaned.tsv** in your submission.

2) Harmonization with target BIM (15 pts)

a. SNP overlap with target

Intersect SNP list in discovery_gwas_sim_sumstats_cleaned.tsv with target .bim SNP IDs; report overlap count and % retained.

SNPs in Discovery cohort: 1,454,675

Overlapping SNPs with target cohort (HapMap): 1,068,130

Percentage Retained: 73.43%

```
import pandas as pd

# -----
# Load Data
# -----

print("Loading datasets for harmonization...")
# 1. Load the cleaned base stats from Step 1
base_df =
pd.read_csv('/Users/kmlanderos/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/d
ata/exercise4/discovery_gwas_sim_sumstats_cleaned.tsv', sep='\t')

initial_base_count = len(base_df)

# 2. Load the target BIM file.
# BIM files have no header and are space/tab separated.
# The standard PLINK 6 columns are: CHR, SNP, CM, BP, A1 (alt), A2 (ref)
bim_cols = ['BIM_CHR', 'SNP', 'BIM_CM', 'BIM_BP', 'BIM_A1', 'BIM_A2']
bim_df =
pd.read_csv('/Users/kmlanderos/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/d
ata/exercise4/HapMap_3_r3_qc_final.bim', sep='\s+', header=None, names=bim_cols)

# -----
# 2a. SNP overlap with target
# -----

# Intersect the base summary stats with the target BIM on the 'SNP' column
merged_df = pd.merge(base_df, bim_df[['SNP', 'BIM_A1', 'BIM_A2']], on='SNP',
how='inner')

overlap_count = len(merged_df)
percent_retained = (overlap_count / initial_base_count) * 100

print(f"2a. SNP Overlap:")
print(f"    - Original base SNPs: {initial_base_count}")
```

```
print(f"    - Overlapping SNPs: {overlap_count}")
print(f"    - % Retained: {percent_retained:.2f}%\n")
```

b. Allele matching

For overlapping SNPs, verify base A1/A2 match target BIM alleles (order may differ).

Of the 1,068,130 overlapping SNPs:

Allele matches (order may differ): 1,062,730

Incompatible alleles: 5,400 -> Removed

(code below in 2c)

c. Allele flipping logic

If base A1/A2 are swapped relative to BIM, flip effect direction: BETA := -BETA and swap alleles.

Report how many SNPs were flipped.

Alleles flipped: 19,229; 1.8% of matching alleles.

```
# -----
# 2b & 2c. Allele matching and flipping logic
# -----
# Condition 1: Exact Match
exact_match = (merged_df['A1'] == merged_df['BIM_A1']) & (merged_df['A2'] ==
merged_df['BIM_A2'])

# Condition 2: Swapped
flip_match = (merged_df['A1'] == merged_df['BIM_A2']) & (merged_df['A2'] ==
merged_df['BIM_A1'])
flipped_count = flip_match.sum()

# Apply the flipping logic:
merged_df.loc[flip_match, 'BETA'] = -merged_df.loc[flip_match, 'BETA']
merged_df.loc[flip_match, 'A1'] = merged_df.loc[flip_match, 'BIM_A1']
merged_df.loc[flip_match, 'A2'] = merged_df.loc[flip_match, 'BIM_A2']

# Discard anything that isn't an exact match or a perfect flip
valid_snps = exact_match | flip_match
invalid_count = len(merged_df) - valid_snps.sum()
merged_df = merged_df[valid_snps].copy()

print(f"2b & 2c. Allele Matching and Flipping:")
print(f"    - Exact allele matches: {exact_match.sum()}")
print(f"    - Alleles swapped & BETAs flipped: {flipped_count}")
print(f"    - Incompatible alleles removed: {invalid_count}\n")
```

d. Ambiguous strand SNP handling

Identify A/T and C/G SNPs and remove them.

82,059 ambiguous A/T or C/G SNPs removed.

Final harmonized SNP count: 980,671

```
# -----
# 2d. Ambiguous strand SNP handling
```

```
# -----
# Identify A/T and C/G pairs
def is_ambiguous(a1, a2):
    alleles = set([a1, a2])
    return alleles == {'A', 'T'} or alleles == {'C', 'G'}

ambiguous_mask = merged_df.apply(lambda row: is_ambiguous(row['A1'], row['A2']),
axis=1)
ambig_count = ambiguous_mask.sum()

# Remove them
merged_df = merged_df[~ambiguous_mask].copy()

print(f"2d. Ambiguous Strand Handling:")
print(f"    - Removed {ambig_count} ambiguous (A/T or C/G) SNPs.\n")
print(f"Final harmonized SNP count: {len(merged_df)}\n")
```

Output a file named `prs_weights_harmonized.tsv` after your QC and harmonization steps with these columns:

SNP — SNP ID (matches the target .bim SNP ID)

A1 — effect allele (aligned to the target allele coding)

Note: This is not the A1 in the "ref allele sense" it is either a risk allele if $\beta > 0$ or a protective allele if $\beta < 0$

BETA — SNP weight (with sign flipped if alleles were swapped)

P — GWAS p-value (handy for thresholding, optional for scoring)

```
final_df = merged_df[['SNP', 'A1', 'BETA', 'P']]

output_file =
'/Users/kmlanderos/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/data/excercise4/prs_weights_harmonized.tsv'
final_df.to_csv(output_file, sep='\t', index=False)
```

Include the `prs_weights_harmonized_qc.tsv` in your submission.

3) PRS-ready output sanity checks (10 pts)

a. Final weight file integrity

Confirm final file `prs_weights_harmonized.tsv` has unique SNP IDs and no missing BETA.

File 'prs_weights_harmonized.tsv' has unique SNPs and no missing BETA

```
# -----
# Load Data
# -----

harmonized_df =
pd.read_csv('/Users/kmlanderos/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/data/excercise4/prs_weights_harmonized.tsv', sep='\t')
```

```

# -----
# 3a. Final weight file integrity
# -----
# 1. Confirm unique SNP IDs
duplicate_count = harmonized_df.duplicated(subset=['SNP']).sum()
is_unique = (duplicate_count == 0)

# 2. Confirm no missing BETAs
missing_beta_count = harmonized_df['BETA'].isna().sum()
no_missing_betas = (missing_beta_count == 0)

print("\n--- 3a. Final Weight File Integrity ---")
print(f"Unique SNP IDs confirmed: {is_unique} (Duplicates found: {duplicate_count})")
print(f"No missing BETAs confirmed: {no_missing_betas} (Missing found: {missing_beta_count})")

```

b. Spot-check allele alignment

Randomly sample ~10 SNPs and manually verify (SNP ID, alleles, flip status) against target BIM.

Everything looks good

```

# -----
# Load Data
# -----

base_df = pd.read_csv('/Users/kmlanderos/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/data/exercise4/discovery_gwas_sim_sumstats_cleaned.tsv', sep='\t')

bim_cols = ['BIM_CHR', 'SNP', 'BIM_CM', 'BIM_BP', 'BIM_A1', 'BIM_A2']
bim_df = pd.read_csv('/Users/kmlanderos/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/data/exercise4/HapMap_3_r3_gc_final.bim', sep='\s+', header=None, names=bim_cols)

# -----
# 3b. Spot-check allele alignment
# -----

print("\n--- 3b. Spot-Check Allele Alignment (n=10) ---")

# Randomly sample 10 SNPs from the harmonized data
sample_snps = harmonized_df.sample(n=10, random_state=42)

# Merge the sample back with the Base and BIM data to create a comparison table

```

```

spot_check_df = sample_snps[['SNP', 'A1', 'BETA']].rename(columns={'A1':
'Harmonized_A1', 'BETA': 'Harmonized_BETA'})
spot_check_df = spot_check_df.merge(base_df[['SNP', 'A1', 'A2',
'BETA']].rename(columns={'A1': 'Base_A1', 'A2': 'Base_A2', 'BETA': 'Base_BETA'}),
on='SNP')
spot_check_df = spot_check_df.merge(bim_df[['SNP', 'BIM_A1', 'BIM_A2']], on='SNP')

# Reorder columns for easy side-by-side reading
spot_check_df = spot_check_df[['SNP', 'Base_A1', 'Base_A2', 'Base_BETA', 'BIM_A1',
'BIM_A2', 'Harmonized_A1', 'Harmonized_BETA']]

# Print the table for manual verification
print(spot_check_df.to_string(index=False))

# -----
# Output Formatting
# -----
# Save the exact same DataFrame to the new required filename
output_file =
'/Users/kmlanderos/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/data/excercis
e4/prs_weights_harmonized_qcd.tsv'
harmonized_df.to_csv(output_file, sep='\t', index=False)
print(f"\nSuccessfully saved final verified file to: {output_file}")

```

Include the **prs_weights_harmonized_qcd.tsv** in your submission.

4) PRS for a Binary Trait (C+T) + Logistic Regression with Ancestry Covariates (80 pts)

Compute PRS in the target cohort using clumping + P-value thresholding (C+T) from cleaned discovery GWAS summary statistics, then test association with a binary phenotype using logistic regression while adjusting for ancestry covariates (PC/MDS).

Required Inputs:

1. Target genotypes:
HapMap_3_r3_1.bed/.bim/.fam files
2. Cleaned base GWAS summary stats:
discovery_gwas_sim_sumstats_cleaned.tsv
3. Binary phenotype file (case/control):
pheno.tsv with columns: FID IID PHENO
where PHENO is 1 = control, 2 = case (PLINK convention)
4. Ancestry covariates:
covar.tsv with columns FID IID PC1 PC2 ... (from GWAS tutorial)

4.1) LD Clumping (Target LD) (20 pts)

To obtain approximately independent SNPs for polygenic risk score (PRS) construction, perform LD clumping using the target genotype dataset.

- a. Using PLINK index SNPs that are in strong LD with an LD threshold of $r = 0.1$ within a 250 kb window.

Required inputs:

1. HapMap_3_r3_1 genotypes

(the command requires you to include the prefix HapMap_3_r3_1 while ensuring all .bed/.bim/.fam files are in the same working directory)

2. discovery_gwas_sim_sumstats_cleaned.tsv

Expected outputs:

1. clump.clumped file

```
# Run clumping in PLINK
! /Users/kmlanderos/Downloads/plink_mac_20250819/plink \
--bfile
/Users/kmlanderos/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/data/exercise
4/HapMap_3_r3_qc_final \
--clump discovery_gwas_sim_sumstats_cleaned.tsv \
--clump-r2 0.1 \
--clump-kb 250 \
--clump-p1 1 \
--clump-p2 1 \
--out clump
```

b. Extract the list of clumped SNPs from clump.clumped and save them in a file called Clumped.snps

87,103 clumped SNPs

```
# Extract the Clumped SNPs
! awk 'NR>1 && NF>0 {print $3}' clump.clumped > clumped.snps
```

Include the **clumped.snps** file in your submission.

4.2) Threshold SNPs by GWAS P-value (20 pts)

After LD clumping, create a PRS SNP sets using p-value thresholding (the p-value here refers to the raw p-values from GWAS). For each threshold T , select SNPs from the cleaned GWAS summary statistics with $p \leq T$, then intersect that set with the SNPs retained after clumping.

Use the following p-value thresholds:

5e-8, 1e-6, 1e-4, 1e-3, 1e-2, 0.05, 0.1, 0.5, 1.0

For each threshold T , identify all SNPs in the cleaned GWAS summary statistics with $P \leq T$, then intersect that SNP list with the set of SNPs retained after LD clumping.

Required inputs:

1. discovery_gwas_sim_sumstats_cleaned.tsv

2. clumped.snps

Expected outputs:

1. For each threshold, generate a file containing the SNPs that:

- pass the specified GWAS P-value cutoff, **and**
- are present in the clumped SNP list
- For example: prs_5e-8.snps, prs_1e-4.snps

Include all **prs_{T}.snps** files in your submission

Number of SNPs per GWAS p-value threshold:

5e-8 = 33 SNPs

1e-6 = 60 SNPs

1e-4 = 232 SNPs

1e-3 = 1,241 SNPs

1e-2 = 8,905 SNPs

0.05 = 27,519 SNPs

0.1 = 39,403 SNPs

0.5 = 71,600 SNPs

1.0 = 87,024 SNPs

```
import pandas as pd

# -----
# Define our thresholds and their exact string representations
# -----
thresholds = {
    "5e-8": 5e-8,
    "1e-6": 1e-6,
    "1e-4": 1e-4,
    "1e-3": 1e-3,
    "1e-2": 1e-2,
    "0.05": 0.05,
    "0.1": 0.1,
    "0.5": 0.5,
    "1.0": 1.0
}

# -----
# Load Data
# -----
print("Loading summary stats and clumped SNPs...")

# 1. Load the cleaned summary statistics
sumstats = pd.read_csv('/Users/kmlander0s/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/data/exercise4/discovery_gwas_sim_sumstats_cleaned.tsv', sep='\t')

# 2. Load the list of clumped SNPs into a Python 'set' for lightning-fast matching
with open('clumped.snps', 'r') as f:
    clumped_snps = set(f.read().splitlines())

print(f"Total SNPs in summary stats: {len(sumstats)}")
print(f"Total SNPs passing LD clumping: {len(clumped_snps)}\n")
```

```

# -----
# Mathematical Intersection (Clumping)
# -----
# Pre-filter the dataframe to only include SNPs that survived the clumping step
# This makes the loop much faster!
clumped_df = sumstats[sumstats['SNP'].isin(clumped_snps)]

# -----
# Thresholding Loop
# -----
print("Generating threshold files...")

for t_str, t_val in thresholds.items():
    # Filter the clumped data for P-values less than or equal to the threshold
    valid_snps = clumped_df[clumped_df['P'] <= t_val]['SNP']

    # Save the SNP IDs to a text file
    out_file = f"/Users/kmlander0s/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/data/exercise4/prs_{t_str}.snps"
    valid_snps.to_csv(out_file, index=False, header=False)

    # Print a status update for your report
    print(f" - {out_file}: {len(valid_snps)} SNPs retained with P <= {t_str}")

```

4.3) Compute PRS with PLINK (20 pts)

For each p-value threshold compute PRS score using PLINK

Required inputs:

1. HapMap_3_r3_1 genotypes
(the command requires you to include the prefix HapMap_3_r3_1 while ensuring all .bed/.bim/.fam files are in the same working directory)
2. pheno.tsv
3. covar.tsv
4. prs_{T}.snps file
5. prs_weights_harmonized_qcd.tsv

Expected outputs:

1. .profile output file per individual

Include all **.profile files** in your submission

```

%%bash

# Define the 9 thresholds in a list
for T in "5e-8" "1e-6" "1e-4" "1e-3" "1e-2" "0.05" "0.1" "0.5" "1.0"
do
    echo "Calculating PRS for threshold: $T"

```

```

/Users/kmlanderos/Downloads/plink_mac_20250819/plink \
--bfile
/Users/kmlanderos/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/data/exercise
4/HapMap_3_r3_qc_final \
--score
/Users/kmlanderos/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/data/exercise
4/prs_weights_harmonized_qcd.tsv 1 2 3 header \
--extract
/Users/kmlanderos/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/data/exercise
4/prs_${T}.snps \
--out prs_${T}
done

```

4.4) Logistic Regression: phenotype ~ covariates + PRS (20 pts)

Next, test whether the PRS is associated with the **binary phenotype** while adjusting for ancestry covariates. For each threshold, use the PRS computed in the previous step as an additional covariate in a **logistic regression model**.

This requires combining:

- the phenotype file
- the ancestry covariate file
- the PRS values from the corresponding .profile file

You will first need to extract the relevant PRS column from the PLINK profile output and merge it with the ancestry covariates so that each individual has:

- FID
- IID
- ancestry PCs (or MDS covariates)
- PRS

Then run logistic regression in PLINK using the binary phenotype as the outcome.

Required inputs:

1. HapMap_3_r3_1.bed/.bim/.fam
2. pheno.tsv
3. covar.tsv
4. prs_{T}.profile

Expected outputs:

1. assoc_\${T}.assoc.logistic

Include all **assoc_\${T}.assoc.logistic** files in your submission

Create 'pheno.tsv' file:

```

# ----- Create pheno.tsv -----

```

```
# 1. Create the file and add the required header
!          echo          -e          "FID\tIID\tPHENO"          >
/Users/kmlanderos/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/data/exercise
4/pheno.tsv

# 2. Extract columns 1, 2, and 6 from the .fam file and append them
!          awk          '{print          $1          "\t"          $2          "\t"          $6}'
/Users/kmlanderos/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/data/exercise
4/HapMap_3_r3_qc_final.fam          >>
/Users/kmlanderos/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/data/exercise
4/pheno.tsv

!          head
/Users/kmlanderos/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/data/exercise
4/pheno.tsv
```

Create merged covariate files:

```
# ----- Create the Merged Covariate Files -----

# 1. Load your base covariates (handles tabs or spaces automatically)
covar_file =
'/Users/kmlanderos/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/data/exercis
e4/covar_mds.csv'
covar = pd.read_csv(covar_file, sep=',')

# 2. Define the thresholds
thresholds = ["5e-8", "1e-6", "1e-4", "1e-3", "1e-2", "0.05", "0.1", "0.5", "1.0"]

print("Merging PRS scores with ancestry covariates...")

# 3. Loop through and create a custom covariate file for each threshold
for T in thresholds:
    # Load the specific PRS profile and grab only the IDs and the SCORE
    prs =
pd.read_csv(f'/Users/kmlanderos/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/
data/exercise4/prs_{T}.profile', sep='\s+')[['FID', 'IID', 'SCORE']]

    # Merge them together, matching perfectly on FID and IID
    merged = pd.merge(covar, prs, on=['FID', 'IID'])

    # Save as a temporary text file formatted for PLINK
```

```

                                out_name
                                =
f'/Users/kmlanderos/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/data/excerci
se4/temp_covar_prs_{T}.txt'
merged.to_csv(out_name, sep='\t', index=False)

print(f" - Successfully created: {out_name}")

print("\nCovariate files are ready for PLINK.")

```

Run Logistic Regressions:

```

%%bash

# ----- Run the Logistic Regression -----

# Define all your absolute paths
WORK_DIR="/Users/kmlanderos/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/data
/excercise4/"
PLINK="/Users/kmlanderos/Downloads/plink_mac_20250819/plink"
BFILE="/Users/kmlanderos/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/data/ex
cercise4/HapMap_3_r3_qc_final"
PHENO_FILE="/Users/kmlanderos/Documents/Johns_Hopkins/Spring_2026/Annotate_a_Genome/da
ta/excercise4/pheno.tsv"

# Loop through all 9 thresholds
for T in "5e-8" "1e-6" "1e-4" "1e-3" "1e-2" "0.05" "0.1" "0.5" "1.0"
do
    echo "======"
    echo "Running Logistic Regression for: $T"
    echo "======"

    $PLINK \
        --bfile $BFILE \
        --pheno $PHENO_FILE \
        --covar ${WORK_DIR}temp_covar_prs_${T}.txt \
        --logistic hide-covar \
        --out ${WORK_DIR}assoc_${T}

Done

```

Link to Code:

https://github.com/KevinMLanderos/Annotate_a_Genome_2026/tree/main/excercise4_GWAS